

TASKS / MATERIAL_ALLOCATION -- PLAYWRIGHT AUTOMATION OVER WFX

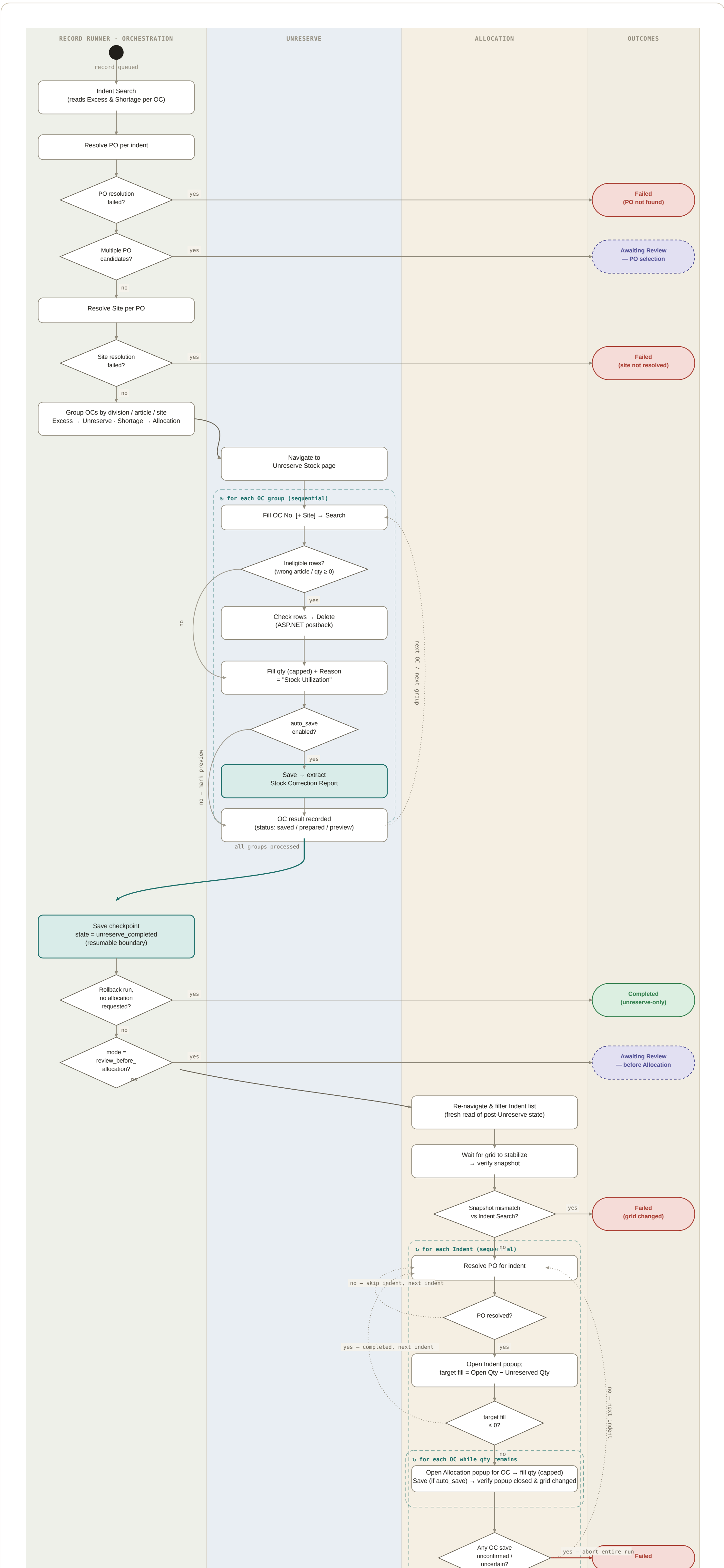
One record, two objects: *Unreserve* hands off to *Allocation*

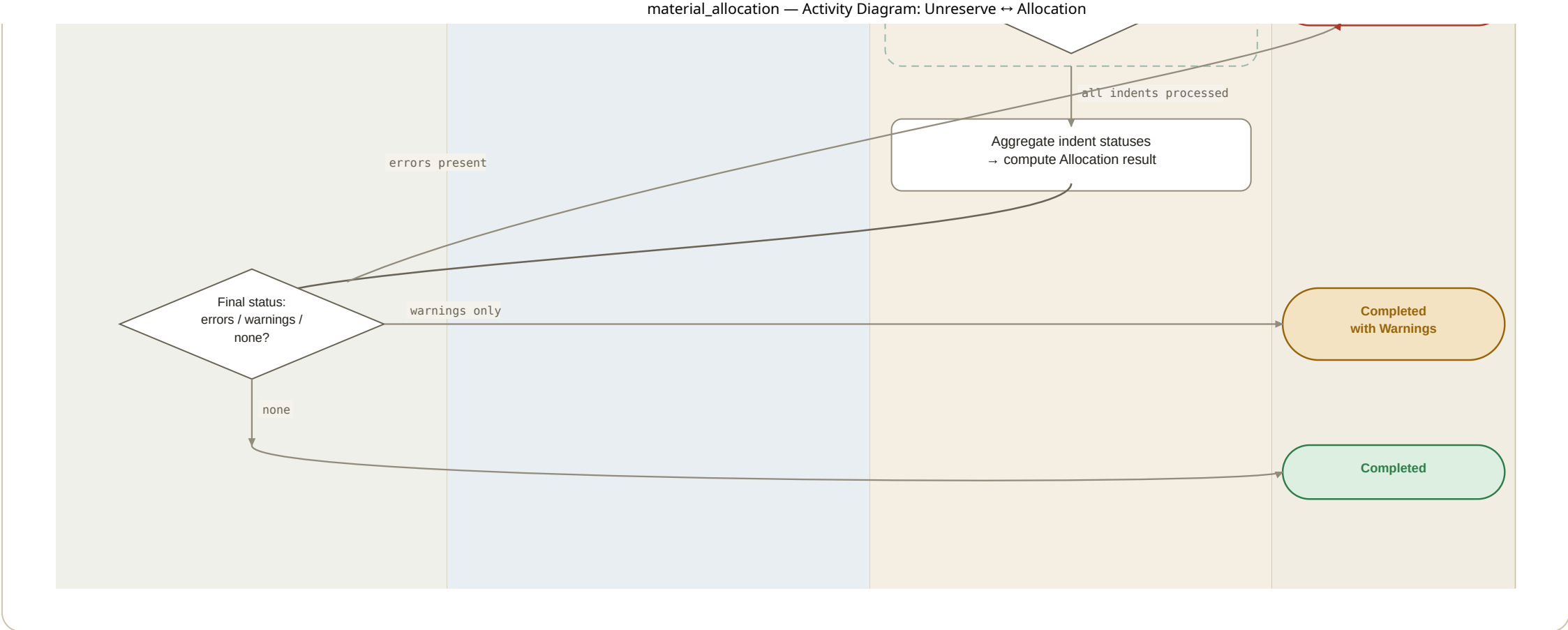
This is the UML activity flow the **PreserveRunner** / **RecordRunner** orchestration actually executes per record. **Unreserve** frees up Excess OCs (negative Open Qty) on the Unreserve Stock page; a checkpoint is written; then **Allocation** re-reads the Indent list and covers Shortage OCs (positive Open Qty) through the Indent -- In-House popups. Both objects sit inside the same sequential per-record loop -- Unreserve always runs first, and the record can only skip straight to Allocation by resuming from that checkpoint.

✖ The teal-bordered box and arrow mark the **one deterministic handoff** between the two objects: record_runner.py:544-553 saves state = unreserve_completed right after the last Unreserve group and before Allocation starts. That save is what makes a crashed or interrupted run resumable without re-doing Unreserve.

READING THE DIAGRAM

- Record queued (start)
- Activity -- a step the automation performs
- ◇ Decision -- a branch in the code
- ⬇ Loop region -- "for each OC group / indent / OC"
- ⏸ Awaiting Review -- paused, resumable via /approve
- ⬆ Terminal state -- run ends here
- ⬆ The Unreserve -- Allocation checkpoint boundary
- ⋯ Loop-back -- next iteration





Swimlanes read left-to-right: **Record Runner** owns indent search, PO/site resolution, the checkpoint, and the final status rollout; **Unreserve** and **Allocation** each own their page-automation loop; **Outcomes** collects every terminal or paused state the record can land in. Detail below the level of "open a popup, fill a field, verify the save" is compressed into single activities — the per-row column resolution, dialog handling, and Stock Correction Report parsing inside Unreserve, and the per-OC In-House popup mechanics inside Allocation, are each real multi-step functions in `unreserve/` and `allocation/` but aren't separate branches this diagram needs to carry.

Sources — `task.py:182` (`route-PreserveRunner`) · `orchestration/preserve_runner.py:33-178` (`run loop`) · `orchestration/record_runner.py:60-673` (`per-record state machine`) · `operations.py:288-291` (`run_unreserve / run_allocation`) · `unreserve/*.py` · `allocation/*.py` · `history/store.py:509-559` (`checkpoint save`) · `contracts/enums.py` (`RecordState`, `PreserveNode`)